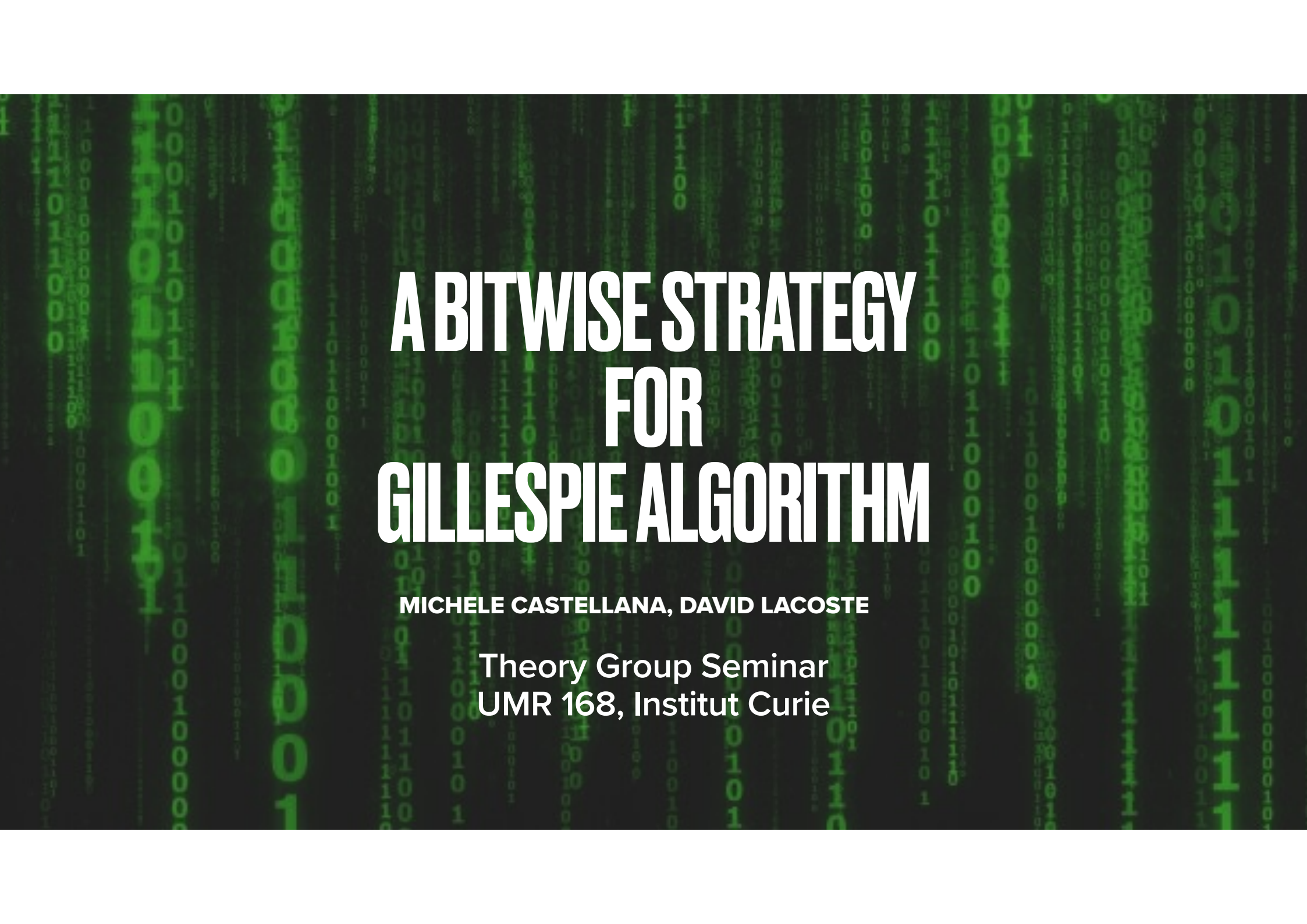




# A BITWISE STRATEGY FOR GILLESPIE ALGORITHM

MICHELE CASTELLANA, DAVID LACOSTE

Theory Group Seminar  
UMR 168, Institut Curie

# OUTLINE

- Homochirality, Frank model
- Gillespie algorithm
- Bitwise Gillespie algorithm
- Gain in numerical yield
- Discussion

# FRANK MODEL

Frank, F. C. (1953), *Biochim. et Biophys. Acta*, **11**:459–463.

- Three particle species:
  - Chiral species:  $L, D$
  - Achiral species:  $A$  (fuel)
- Three reactions
  - $A + D \rightarrow 2D$
  - $A + L \rightarrow 2L$
  - $D + L \rightarrow 2A$
- Homochirality: biomolecules exist in a single chiral state
- Model for homochirality
- Multiple fixed points (racemic, homochiral)

# FRANK MODEL

Frank, F. C. (1953), *Biochim. et Biophys. Acta*, **11**:459–463.

- **Three particle species:**
  - Chiral species: L, D
  - Achiral species: A (fuel)
- **Three reactions**
  - $A + D \rightarrow 2D$
  - $A + L \rightarrow 2L$
  - $D + L \rightarrow 2A$

Definitions:

- Particle numbers:  $x_L, x_D, x_A$
- $\mathbf{x} \equiv \{x_L, x_D, x_A\}$
- Reaction probabilities:  $a_j(\mathbf{x}), j = 1, 2, 3$ 
  - $a_1(\mathbf{x}) = x_A x_D$
  - $a_2(\mathbf{x}) = x_A x_L$
  - $a_3(\mathbf{x}) = x_D x_L$
- State-change vectors  $\mathbf{v}_j, j = 1, 2, 3$ 
  - $\mathbf{v}_1 = \{-1, +1, 0\}$
  - $\mathbf{v}_2 = \{-1, 0, +1\}$
  - $\mathbf{v}_3 = \{-1, -1, +2\}$

# GILLESPIE ALGORITHM

Gillespie D.T. (1976) *J. Comput. Phys.*  
**22**:403.

- Captures fluctuations for small particle numbers
- Large particle numbers  $\rightarrow$  reaction-diffusion equations
- Simple

# GILLESPIE ALGORITHM

Definitions:

- Particle numbers:  $x_L, x_D, x_A$
- $\mathbf{x} \equiv \{x_L, x_D, x_A\}$
- Reaction probabilities:  $a_j(\mathbf{x}), j = 1, 2, 3$

$$a_1(\mathbf{x}) = x_A x_D$$

$$a_2(\mathbf{x}) = x_A x_L$$

$$a_3(\mathbf{x}) = x_D x_L$$

$$\text{Normalization: } z(\mathbf{x}) \equiv \sum_j \mathbf{a}_j(\mathbf{x})$$

- State-change vectors  $\mathbf{v}_j, j = 1, 2, 3$

$$\mathbf{v}_1 = \{-1, +1, 0\}$$

$$\mathbf{v}_2 = \{-1, 0, +1\}$$

$$\mathbf{v}_3 = \{-1, -1, +2\}$$

Algorithmic steps

1. Evaluate all  $a_j(\mathbf{x})$  and  $z(\mathbf{x})$
2. Draw the time  $\tau$  for the reaction  
 $p(\tau) \sim \exp(-z(\mathbf{x}) \tau)$
3. Draw the reaction index  $j$   
 $p(j) \sim a_j(\mathbf{x})$
4. Effect the reaction:
  - $t \rightarrow t + \tau$
  - $\mathbf{x} \rightarrow \mathbf{x} + \mathbf{v}_j$
5. Go to 1.

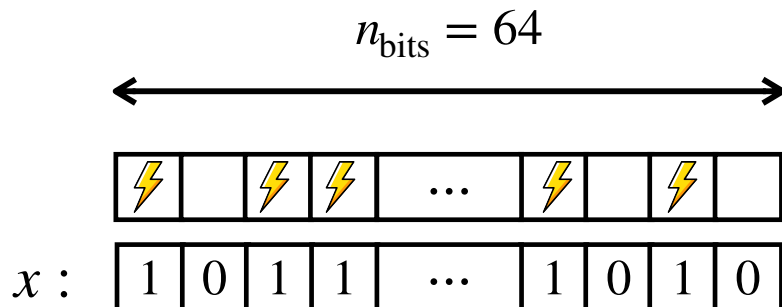
# BITWISE REPRESENTATION

Particle number  $x$

$$1. \text{ Binary representation: } x = \sum_{i=1}^{n_{\text{bits}}} x_i 2^i, \\ x_i = \{0,1\}$$

2. 64-bit unsigned integer type

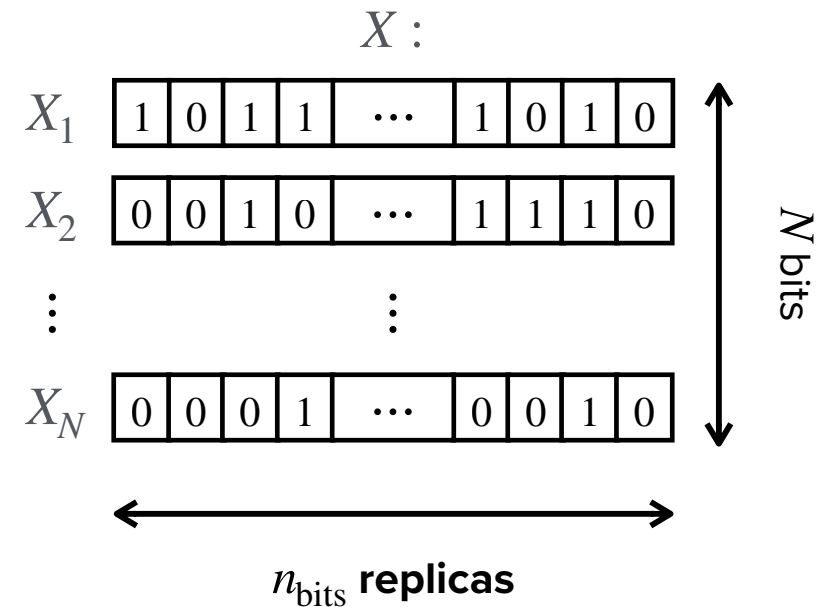
`unsigned long long int`  $x = 254923137894571039;$



**Multi-particle number**  $X$

$$X = \sum_{i=1}^N X_i 2^i, X_i =$$

|   |   |   |   |     |   |   |   |   |
|---|---|---|---|-----|---|---|---|---|
| 1 | 0 | 1 | 1 | ... | 1 | 0 | 1 | 0 |
|---|---|---|---|-----|---|---|---|---|



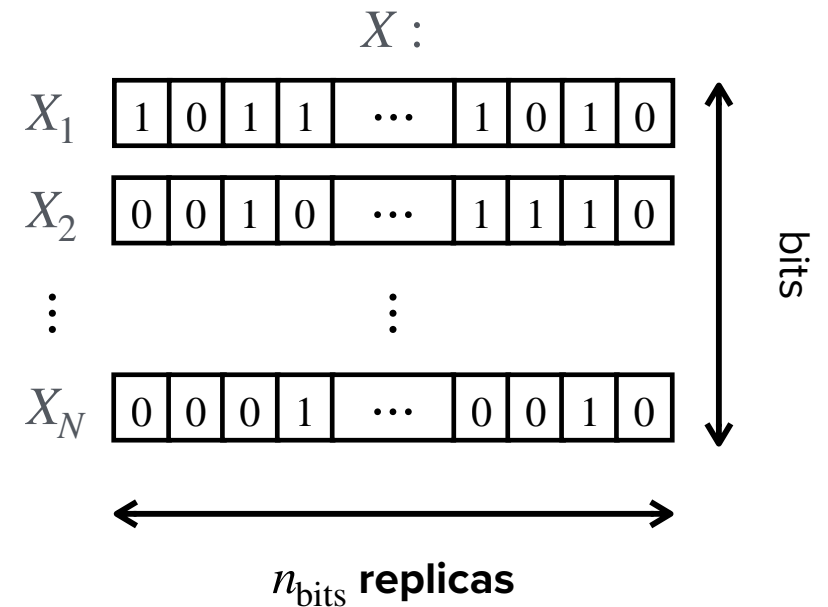
# BITWISE REPRESENTATION

- $X$  represents  $n_{\text{bits}}$  physical Frank models
- **Advantage: simulate  $n_{\text{bits}}$  replicas at the same time** [M. Palassini and S. Caracciolo, *Phys. Rev. Lett.*, 82(25):5128, 1999.]

**Gain in computational yield?**

**Multi-particle number  $X$**

$$X = \sum_{i=1}^N X_i 2^i, X_i = \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 1 & 0 & 1 & 1 & \cdots & 1 & 0 & 1 & 0 \\ \hline \end{array}$$





# BITWISE REPRESENTATION

Gillespie algorithm:

- Particle numbers:  $x_L, x_D, x_A$
- $\mathbf{x} \equiv \{x_L, x_D, x_A\}$
- Reaction probabilities:  $a_j(\mathbf{x}), j = 1, 2, 3$ 

$$a_1(\mathbf{x}) = x_A x_D$$

$$a_2(\mathbf{x}) = x_A x_L$$

$$a_3(\mathbf{x}) = x_D x_L$$

Normalization:  $z(\mathbf{x}) \equiv \sum_j a_j(\mathbf{x})$
- State-change vectors  $\mathbf{v}_j, j = 1, 2, 3$ 

$$\mathbf{v}_1 = \{-1, +1, 0\}$$

$$\mathbf{v}_2 = \{-1, 0, +1\}$$

$$\mathbf{v}_3 = \{-1, -1, +2\}$$

“Multi-Gillespie” algorithm:

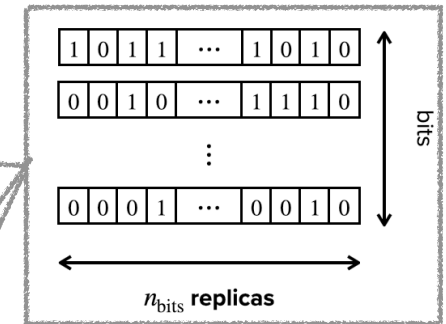
- $X_L, X_D, X_A$
- $\mathbf{X} \equiv \{X_L, X_D, X_A\}$
- $A_j(\mathbf{X})$ 

$$A_1(\mathbf{X}) = X_A \times X_D$$

$$\vdots$$

$$Z(\mathbf{X}) = \sum_j A_j(\mathbf{X})$$
- $V_1 = \{-1, +1, 0\}$ 

$$\vdots$$



# BITWISE REPRESENTATION

```
class UnsignedInt{  
  
private:  
    vector<Bits> b;  
  
public:  
  
    UnsignedInt(unsigned long long int);  
  
    [...]  
    //arithmetic operator overloads  
    UnsignedInt operator + (UnsignedInt*), operator - (UnsignedInt*);  
    void operator += (UnsignedInt*), operator -= (UnsignedInt*), operator *= (UnsignedInt*);  
    Bits operator == (UnsignedInt&), operator < (const UnsignedInt&), operator <= (UnsignedInt&);  
  
};
```

# BITWISE REPRESENTATION

Input

```
UnsignedInt X(10), Y(10);  
X.SetRandom(0);  
Y.SetRandom(1);  
X.Print("X before the sum");  
Y.Print("Y");  
X += &Y;  
X.Print("X after the sum");
```

Output

X before the sum

|     |   |   |                                                                   |   |
|-----|---|---|-------------------------------------------------------------------|---|
| [0] | = | { | 0010001010001111100011001100111001010010001101101101011101111001  | } |
| [1] | = | { | 11011101001110001111110010010011111110011001001011011101110000100 | } |
| [2] | = | { | 1010011000001101100010010011010011011100010000010111011101010010  | } |
| [3] | = | { | 0110001101001010111001110000100100010001101100110101101110011111  | } |

Y

|     |   |   |                                                                     |   |
|-----|---|---|---------------------------------------------------------------------|---|
| [0] | = | { | 0001100000011110111101010100111100111101000100101100000101000110    | } |
| [1] | = | { | 1001101111111100000000111001100110100110101000110011111101111011    | } |
| [2] | = | { | 1011101001001111110110010001000000000100010010100011010000001011    | } |
| [3] | = | { | 1100011100011000101111100111001110110111011011011000100111000111110 | } |

X after the sum

|     |   |   |                                                                  |   |
|-----|---|---|------------------------------------------------------------------|---|
| [0] | = | { | 0011101010010001011110011000000101101111001001000001011000111111 | } |
| [1] | = | { | 010001101100001101111101111000001000101100101000100010110111111  | } |
| [2] | = | { | 100001010111111011010001001010110110101000101000111110000011001  | } |
| [3] | = | { | 000111100101111110100000110111000110010100111000010001011100011  | } |
| [4] | = | { | 111000110000100010101110001000110010101011000110101111100011110  | } |

$n_{\text{bits}}$  replicas

bits

1 + 2 = 3      14 + 7 = 21      5 + 3 = 8

# BITWISE GILLESPIE ALGORITHM

Implementation:

- Same  $\tau$  for all replicas
- Initialize all replicas with **random, independent initial conditions**
- **No correlations**

Algorithmic steps:

1. Evaluate all  $A_j(\mathbf{X})$  and  $Z(\mathbf{X}) = \sum_j A_j(\mathbf{X})$
2. Draw the time  $\tau$  for the reaction  
 $p(\tau) \sim \exp(-Z(\mathbf{X}) \tau)$
3. Draw the reaction index  $j$   
 $p(j) \sim A_j(\mathbf{X})$
4. Effect the reaction:
  - $t \rightarrow t + \tau$
  - $\mathbf{X} \rightarrow \mathbf{X} + \mathbf{V}_j$
5. Go to 1.

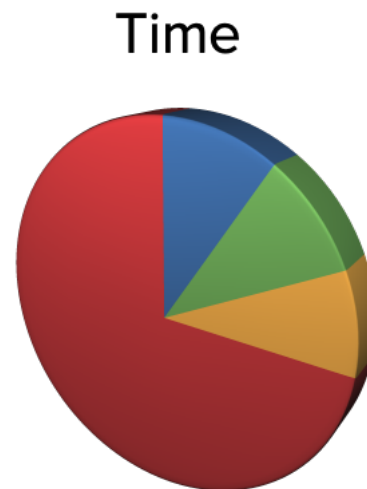
**Gain in computational yield?**

# COMPUTATIONAL GAIN

**Standard** Gillespie algorithm:

Algorithmic steps

1. Evaluate all  $a_j(\mathbf{x})$  and  $z(\mathbf{x}) = \sum_j a_j(\mathbf{x})$
2. Draw the time  $\tau$  for the reaction
  1.  $p(\tau) \sim \exp(-z(\mathbf{x}) \tau)$
3. Draw the reaction index  $j$ 
  1.  $p(j) \sim a_j(\mathbf{x})$
4. Store  $\mathbf{x}$
5. Effect the reaction:
  1.  $t \rightarrow t + \tau$
  2.  $\mathbf{x} \rightarrow \mathbf{x} + \mathbf{v}_j$
6. Go to 1.



Bottleneck = data storage + random-number generation (2, 3 and 4)



**Use the same random numbers for all replicas in bitwise-Gillespie**

# COMPUTATIONAL GAIN

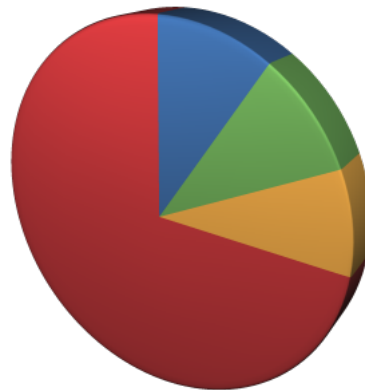
Arithmetic operations highly optimized → Lose on 1 and 5

**Standard** Gillespie algorithm:

Algorithmic steps

1. Evaluate all  $a_j(\mathbf{x})$  and  $z(\mathbf{x}) = \sum_j a_j(\mathbf{x})$
2. Draw the time  $\tau$  for the reaction
  1.  $p(\tau) \sim \exp(-z(\mathbf{x}) \tau)$
3. Draw the reaction index  $j$ 
  1.  $p(j) \sim a_j(\mathbf{x})$
4. Store  $\mathbf{x}$
5. Effect the reaction:
  1.  $t \rightarrow t + \tau$
  2.  $\mathbf{x} \rightarrow \mathbf{x} + \mathbf{v}_j$
6. Go to 1.

Time

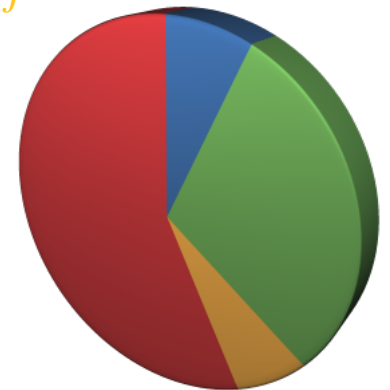


**Bitwise** Gillespie algorithm:

Algorithmic steps:

1. Evaluate all  $A_j(\mathbf{X})$  and  $Z(\mathbf{X}) = \sum_j A_j(\mathbf{X})$
2. Draw the time  $\tau$  for the reaction
  1.  $p(\tau) \sim \exp(-Z(\mathbf{X}) \tau)$
3. Draw the reaction index  $j$ 
  1.  $p(j) \sim A_j(\mathbf{X})$
4. Store  $\mathbf{X}$
5. Effect the reaction:
  1.  $t \rightarrow t + \tau$
  2.  $\mathbf{X} \rightarrow \mathbf{X} + \mathbf{V}_j$
6. Go to 1.

Time



# COMPUTATIONAL GAIN

Arithmetic operations highly optimized → Lose on 1 and 5

**Standard** Gillespie algorithm:

|       |   |   |   |
|-------|---|---|---|
| 1     | 0 | 1 | + |
|       | 1 | 1 | = |
| <hr/> |   |   |   |
|       |   |   |   |



**Bitwise** Gillespie algorithm:

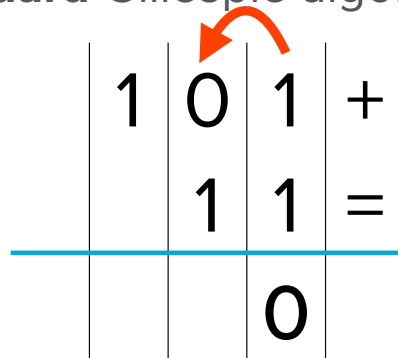


# COMPUTATIONAL GAIN

Arithmetic operations highly optimized → Lose on 1 and 5

**Standard** Gillespie algorithm:

|       |   |   |   |
|-------|---|---|---|
| 1     | 0 | 1 | + |
|       | 1 | 1 | = |
| <hr/> |   |   |   |
|       |   | 0 |   |





**Bitwise** Gillespie algorithm:





# COMPUTATIONAL GAIN

Arithmetic operations highly optimized → Lose on 1 and 5

**Standard** Gillespie algorithm:

|       |   |   |   |
|-------|---|---|---|
| 1     | 0 | 1 | + |
|       | 1 | 1 | = |
| <hr/> |   |   |   |
|       | 0 | 0 |   |





**Bitwise** Gillespie algorithm:



# COMPUTATIONAL GAIN

Arithmetic operations highly optimized → Lose on 1 and 5

**Standard** Gillespie algorithm:



|       |   |   |   |
|-------|---|---|---|
| 1     | 0 | 1 | + |
|       | 1 | 1 | = |
| <hr/> |   |   |   |
| 0     | 0 | 0 |   |



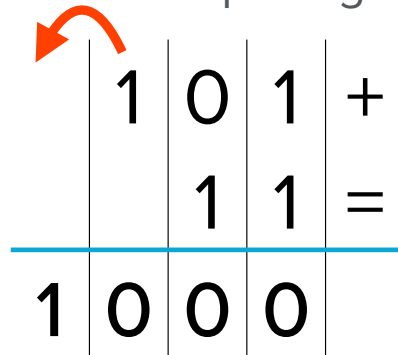
**Bitwise** Gillespie algorithm:



# COMPUTATIONAL GAIN

Arithmetic operations highly optimized → Lose on 1 and 5

**Standard** Gillespie algorithm:



A diagram illustrating binary addition. The first row contains the digits 1, 0, 1 followed by a plus sign. The second row contains the digits 1, 1 followed by an equals sign. A horizontal blue line is drawn below the second row. The third row contains the result digits 1, 0, 0, 0. An orange curved arrow points from the first column (the leftmost '1' in the first row) to the first column of the result row.

|       |   |   |   |
|-------|---|---|---|
| 1     | 0 | 1 | + |
|       | 1 | 1 | = |
| <hr/> |   |   |   |
| 1     | 0 | 0 | 0 |



**Bitwise** Gillespie algorithm:

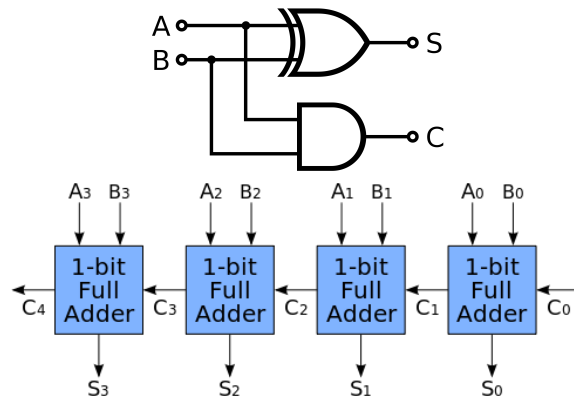


# COMPUTATIONAL GAIN

Arithmetic operations highly optimized → Lose on 1 and 5

**Standard** Gillespie algorithm:

$$\begin{array}{r} 101 \\ + 11 \\ \hline 1000 \end{array}$$



**Bitwise** Gillespie algorithm:

X before the sum

```
[0] = { 00100010 1001111100011001100111001010010001101101101110111001 }
[1] = { 11011101 0011000111111001001011111100110010110111011000100 }
[2] = { 10100110 0001101100010010011010011011100010000010111011010010 }
[3] = { 01100011 0100101011100111000100100010001101100110110111011111 }
```

Y

```
[0] = { 00011000 000111101111010101001111001100100101100000101000110 }
[1] = { 10011011 111111000000001110011001101001101010001100111110111011 }
[2] = { 10111010 01001111101100100010000000010001001010001101000001011 }
[3] = { 11000111 00011000101111101111001110110111011011000100111000111110 }
```

X after the sum

```
[0] = { 00111010 10010001011110011000000101101111001001000001011000111111 }
[1] = { 01000110 1100001101111110111100000100010110010100010001011011111 }
[2] = { 10000101 0111111011010001001010110110101000101000111110000011001 }
[3] = { 00011110 0101111110100000110111000110010100111000010001011100011 }
[4] = { 11100011 0001000101011110001000110010101011000110101111100111110 }
```

bits

$n_{\text{bits}}$  replicas

1 + 2 = 3      14 + 7 = 21      5 + 3 = 8



# COMPUTATIONAL GAIN

Arithmetic operations highly optimized → Gain on 2, 3 and 4

**Standard** Gillespie algorithm:

Algorithmic steps

1. Evaluate all  $a_j(\mathbf{x})$  and  $z(\mathbf{x}) = \sum_j a_j(\mathbf{x})$
2. Draw the time  $\tau$  for the reaction
  1.  $p(\tau) \sim \exp(-z(\mathbf{x}) \tau)$
3. Draw the reaction index  $j$ 
  1.  $p(j) \sim a_j(\mathbf{x})$
4. Store  $\mathbf{x}$
5. Effect the reaction:
  1.  $t \rightarrow t + \tau$
  2.  $\mathbf{x} \rightarrow \mathbf{x} + \mathbf{v}_j$
6. Go to 1.



**Bitwise** Gillespie algorithm:

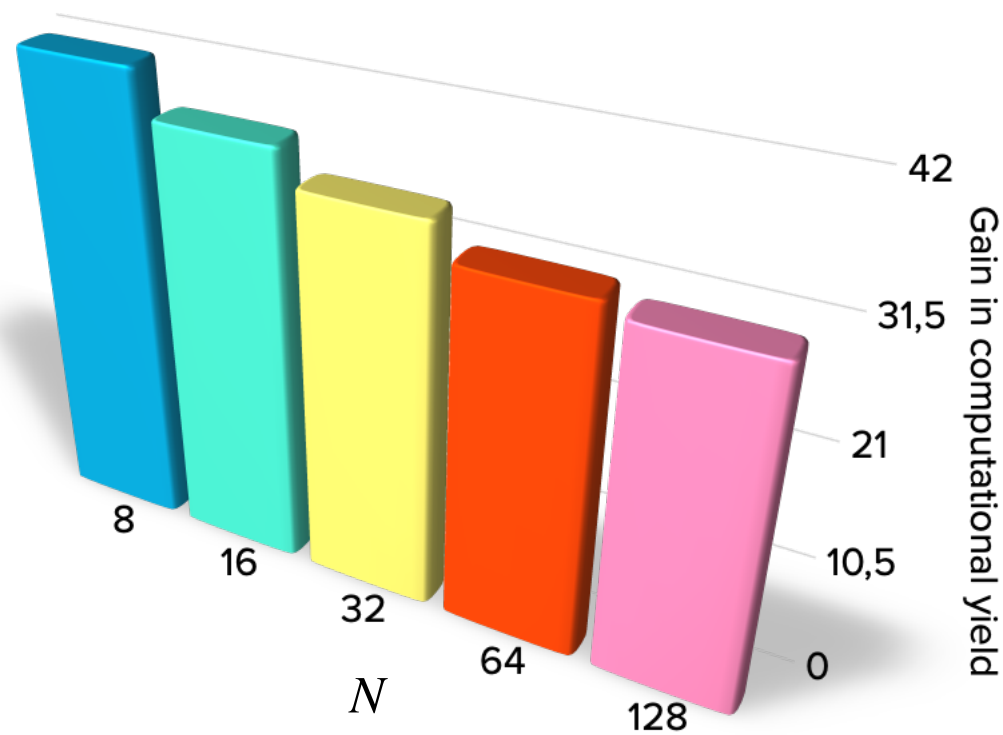
Algorithmic steps:

1. Evaluate all  $A_j(\mathbf{X})$  and  $Z(\mathbf{X}) = \sum_j A_j(\mathbf{X})$
2. Draw the time  $\tau$  for the reaction
  1.  $p(\tau) \sim \exp(-Z(\mathbf{X}) \tau)$
3. Draw the reaction index  $j$ 
  1.  $p(j) \sim A_j(\mathbf{X})$
4. Store  $\mathbf{X}$
5. Effect the reaction:
  1.  $t \rightarrow t + \tau$
  2.  $\mathbf{X} \rightarrow \mathbf{X} + \mathbf{V}_j$
6. Go to 1.



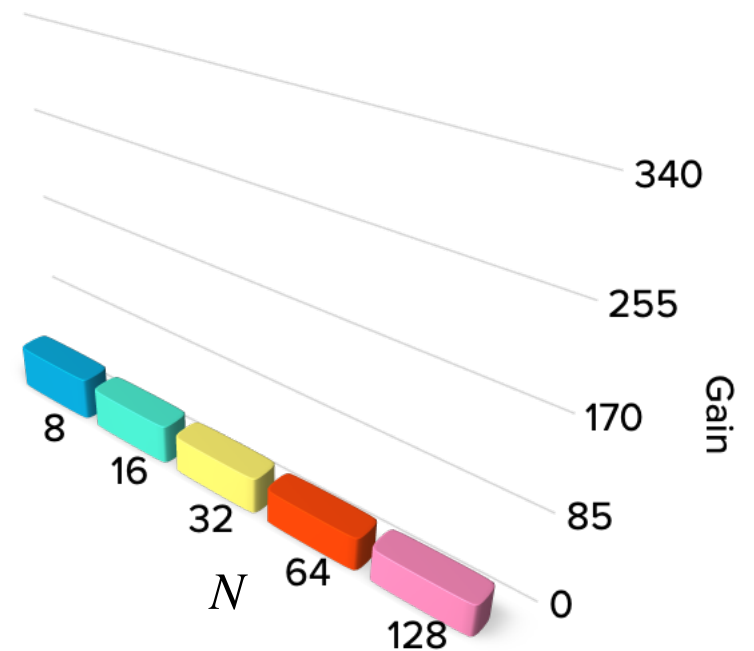
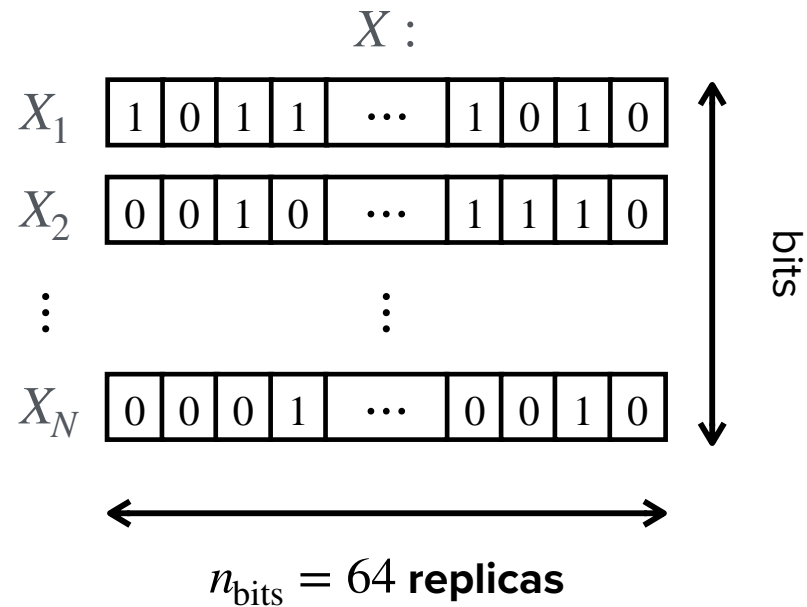
# GAIN IN COMPUTATIONAL YIELD

unsigned long long int  $\times (n_{\text{bits}} = 64)$



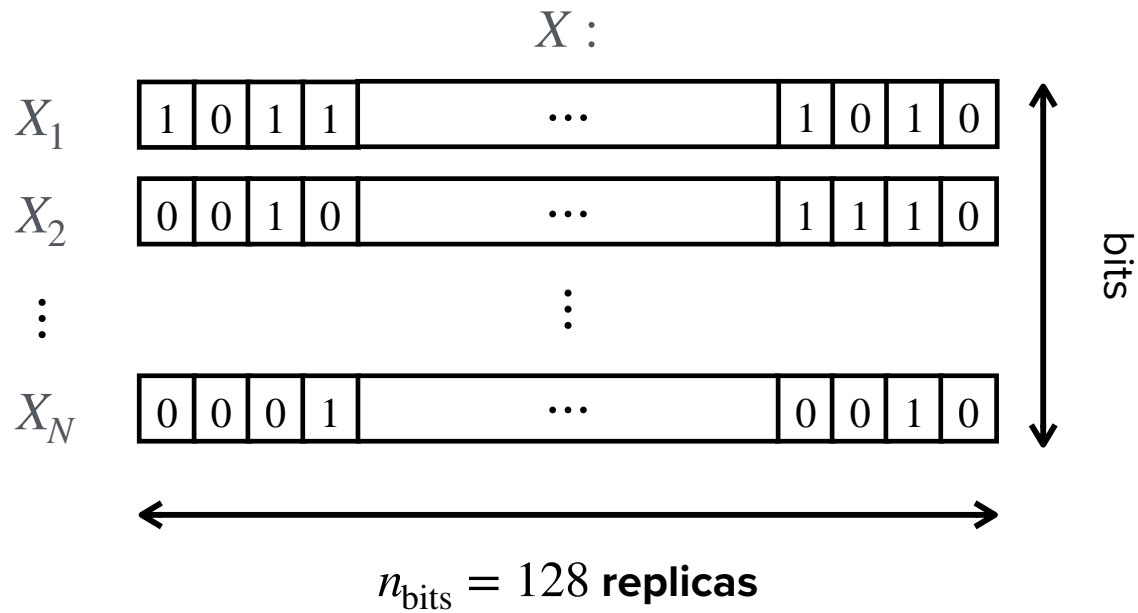
# GAIN IN COMPUTATIONAL YIELD

unsigned long long int  $\times (n_{\text{bits}} = 64)$



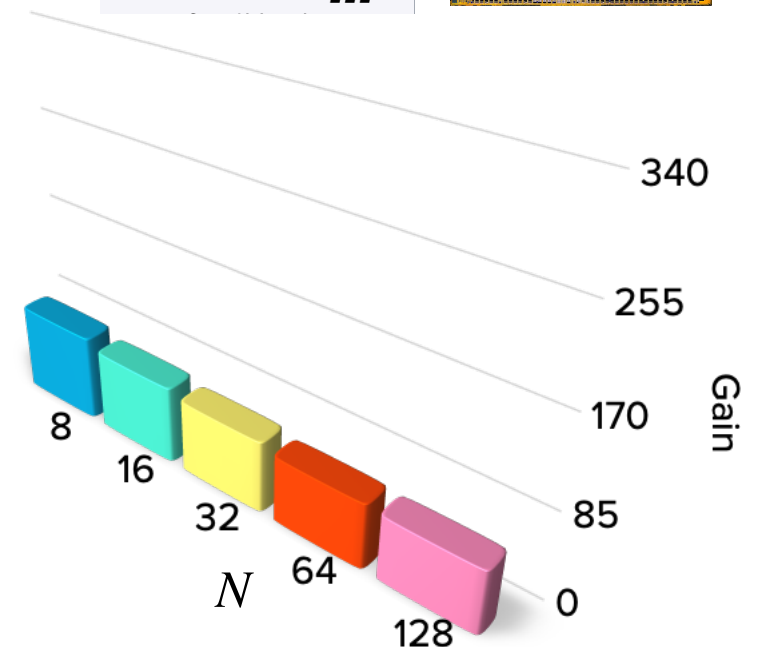
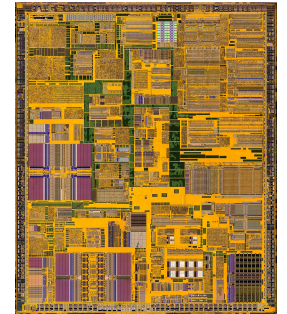
# GAIN IN COMPUTATIONAL YIELD

`__m128` × ( $n_{\text{bits}} = 128$ )



```
__m128 x, y, z;
z = mm128_xor_ps(x, y);
```

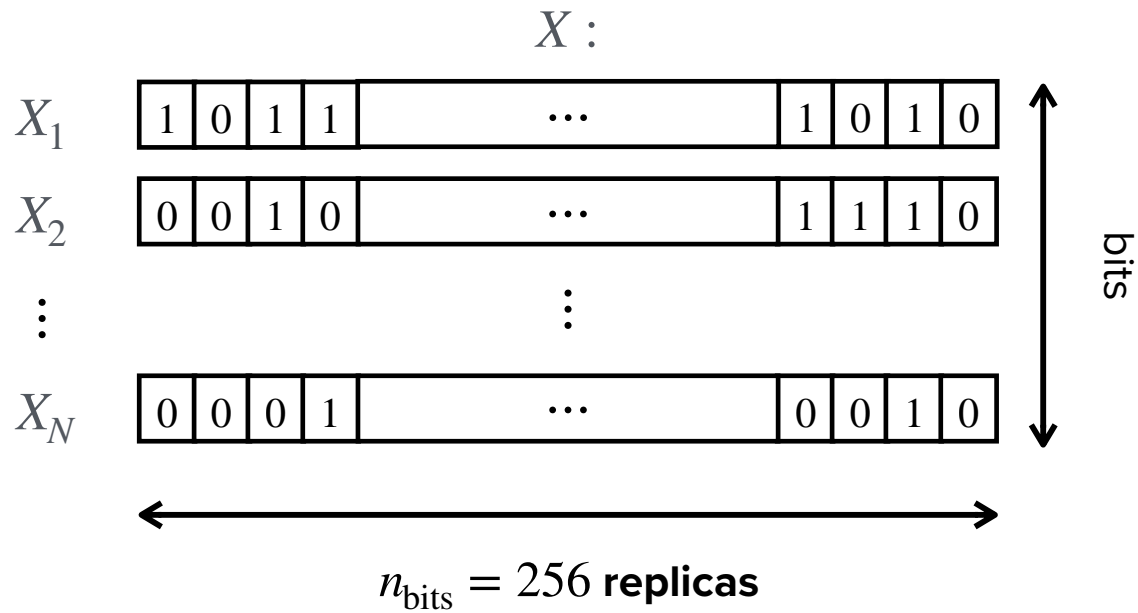
Intel Pentium 3 (1999)





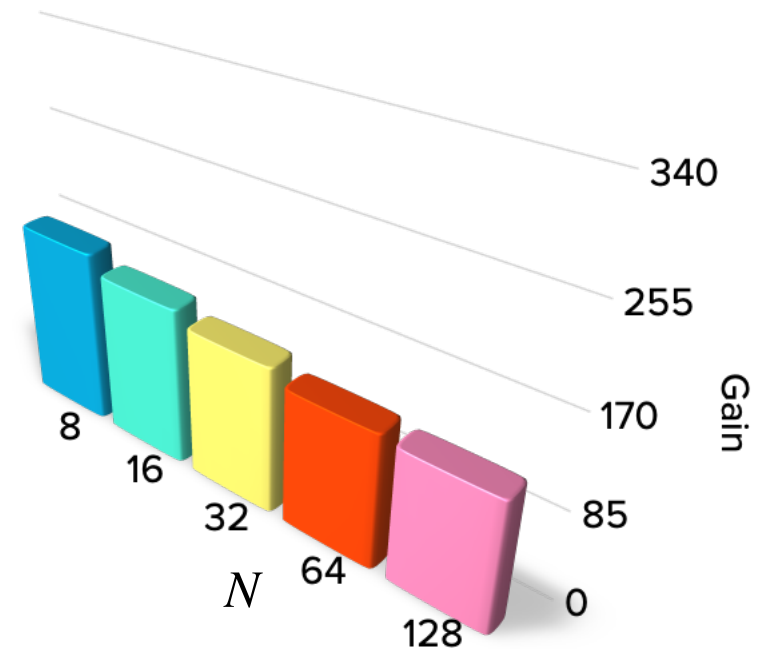
# GAIN IN COMPUTATIONAL YIELD

`__m256` × ( $n_{\text{bits}} = 256$ )



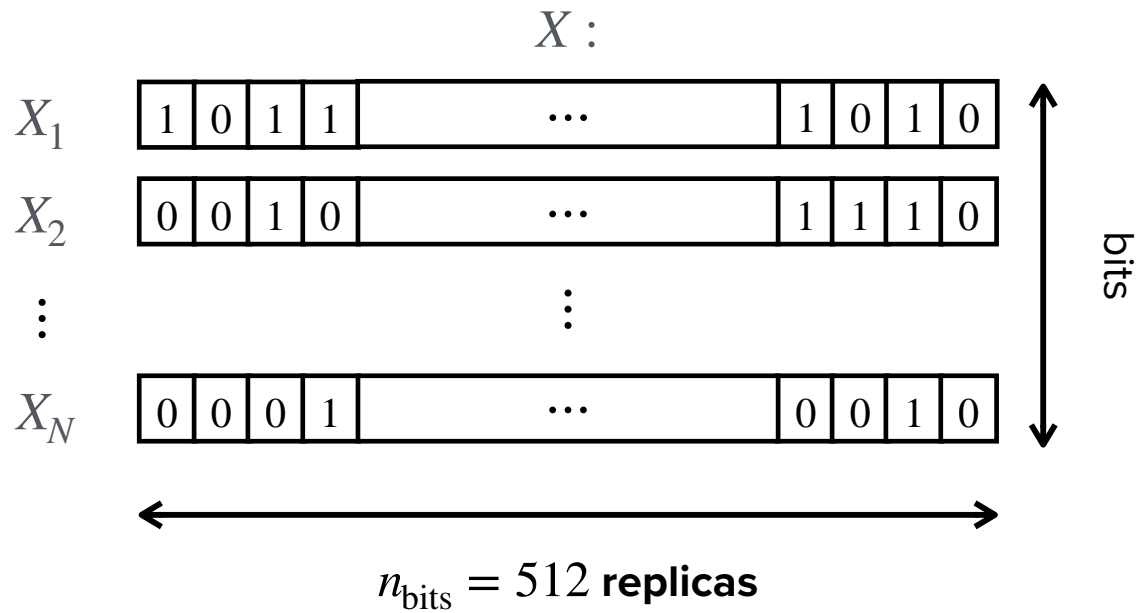
`__m128`  $x, y, z;$   
 $z = \text{mm256\_xor\_ps}(x, y);$

Intel Sandy Bridge (2011)



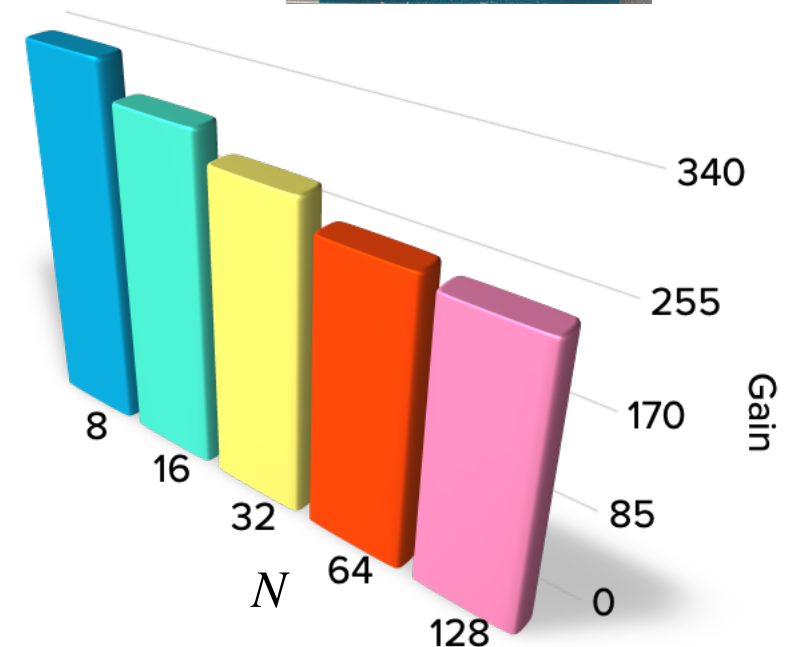
# GAIN IN COMPUTATIONAL YIELD

`__m512` × ( $n_{\text{bits}} = 512$ )



```
__m512 x, y, z;
z = mm512_xor_ps(x, y);
```

Intel Knights Landing (2013)



# OUTLINE

- Frank model for homochirality
- Bitwise Gillespie algorithm
  - Independent initial conditions
  - Same random number for all replicas
- **Computational gain**  $\sim 30 - 330 \times$

## When is this useful?:

- **Sampling multiple initial conditions (large deviations)**
- **Instances with bottlenecks (random-number generation, data storage, ...)**

# ACKNOWLEDGMENTS

- D. Lacoste (ESPCI, Paris)
- G. Laurent (ESPCI, Paris)
- V. Martin Mayor (Universidad Complutense de Madrid, Spain)